

PROCESAMIENTO DE DATOS MASIVOS

Actividad 3 – Programación en Hadoop

Ejercicio 3

Enunciado

Dado el siguiente programa Big Data que tiene defectos de diseño, **se debe entender cuál es el defecto y hacer un informe que contenga:** (1) Nombre y apellidos, (2) Tiempo empleado por el alumno en entender cuál es el defecto, y (3) Descripción del defecto. Todos los archivos del programa se encuentran en la carpeta: “PersonasQueCompranEnMuchasTiendas”.

Se tiene que analizar el programa para entender el defecto y posteriormente realizar un informe llamado PersonasQueCompranEnMuchasTiendas.pdf que contenga lo siguiente:

1. Nombre y apellidos del estudiante
2. Tiempo empleado en entender el defecto del programa
3. Descripción del defecto:
 - a) Circunstancias bajo las que falla el programa: se tiene que indicar en qué ejecuciones podría fallar el programa.
 - b) Motivos por los que falla el programa: se tiene que describir qué es lo que tiene erróneo el programa y que lo hace fallar.
 - c) Directrices para corregir el defecto: se tiene que indicar a grandes rasgos lo que tendría que cambiar el equipo de desarrollo para eliminar el defecto del programa. No hace falta desarrollar el programa correcto, pero sí hay que indicar qué se tendría que cambiar.
4. ¿Te fue útil la información de depuración (la imagen reduccion.jpg y que el defecto se encontraba en >1 Combiners) para entender el defecto? Sí/No e indicar el por qué.

Solución

Se propone la solución contenida en la carpeta “ej3”, compuesta por los siguientes documentos:

- Datos de entrada: casoDePrueba1.txt
- Código: se han proporcionado los scripts, MapperPersonasQueCompranEnMuchasTiendas.py, CombinerPersonasQueCompranEnMuchasTiendas.py, ReducerPersonasQueCompranEnMuchasTiendas.py.
- Comando de ejecución: el código a escribir en la terminal se reporta en: terminal_PersonasQueCompranEnMuchasTiendasDefecto.txt y terminal_PersonasQueCompranEnMuchasTiendas.txt
- Resultado: el resultado generado por el programa se reporta en la carpeta: salidaPersonasQueCompranEnMuchasTiendas y salidaPersonasQueCompranEnMuchasTiendasDefecto

Nombre, apellidos y tiempo empleado

Este trabajo ha sido realizado por **Daniel López y Ania Gorostiza**, estudiantes del **Máster en Big Data y Ciencia de Datos**, en el marco de la asignatura de **Procesamiento de Datos Masivos**.

El objetivo de la práctica consistía en analizar el funcionamiento del *framework* **Hadoop MapReduce**, prestando especial atención al uso del **combiner** y su impacto en la integridad y consistencia de los datos procesados en entornos distribuidos.

Durante el desarrollo de la actividad, se nos planteó un problema relacionado con la **pérdida aleatoria de registros** durante la ejecución de los *jobs*, que se producía al forzar la utilización de un *combiner* en contextos donde los datos se particionaban en *splits* reducidos y procesados en paralelo por distintos nodos del clúster. Este error se manifestaba claramente al comparar los resultados obtenidos con y sin *combiner*, observándose una discrepancia significativa en el número de registros de salida.

El tiempo estimado para **comprender la raíz del problema, reproducirlo de forma controlada y analizar su comportamiento** fue de aproximadamente **4 a 5 horas**, distribuidas entre:

- El análisis detallado del log de ejecución del *job*.
- La revisión de las salidas intermedias y finales de las fases *Map* y *Reduce*.
- La realización de pruebas controladas con diferentes configuraciones de *splits*, *combiners* y cantidades de datos para aislar las condiciones bajo las cuales se producía el defecto.

Las principales dificultades surgieron en la **identificación precisa de cuándo y cómo se ejecutaba el *combiner***, y en cómo este afectaba a la integridad de los datos parciales antes de llegar al *reducer*. Adicionalmente, el **comportamiento no determinista** del *combiner* —que depende de la segmentación y distribución física de los datos— dificultó su análisis sistemático, ya que el defecto no se presentaba de forma constante en todas las ejecuciones.

Uno de los aprendizajes clave ha sido comprender que el uso del *combiner* no es trivial y que, para garantizar la validez de los resultados, la operación que realiza debe cumplir las propiedades de **asociatividad y conmutatividad**, permitiendo combinar resultados parciales en cualquier orden y número de veces sin alterar el resultado final. Esta consideración, ampliamente explicada por el profesor durante la asignatura, resultó decisiva para comprender la causa raíz del problema.

Gracias a una combinación de **pruebas controladas, análisis de logs, consulta de documentación oficial y repaso de las sesiones grabadas**, conseguimos entender el funcionamiento interno del proceso y resolver el defecto identificado.

Descripción del defecto

El programa falla **cuando se ejecutan más de un *Combiner* en el clúster Hadoop**. Como Hadoop Streaming permite que se ejecuten varias instancias de Combiner en diferentes nodos o particiones, dependiendo de la configuración y carga del clúster, el programa puede fallar en aquellas ejecuciones en las que:

- Se dividen los datos de una misma persona en diferentes *combiners*.
- Cada *Combiner* realiza una agregación parcial local sin conservar toda la información necesaria para que el *Reducer* final pueda tomar una decisión correcta.

Esto genera inconsistencias: en unas ejecuciones el resultado es correcto (cuando una persona y todas sus compras van al mismo *Combiner*), y en otras no (cuando se reparten entre diferentes *Combiners*).

Ejemplo concreto:

Supongamos que tenemos este dataset:

```
nginx
Alice NuncCorp
Alice ArcuCompany
Alice PharetraCompany
```

Si los 3 registros se procesan en un solo combiner o sin combiner, no hay problema. Pero si:

- El **combiner 1** recibe:

```
nginx
Alice NuncCorp
Alice ArcuCompany
```

- El **combiner 2** recibe:

```
nginx
Alice PharetraCompany
```

Si el *Combiner* local ya decide si se supera o no el umbral de 3 tiendas distintas, **ninguno de los dos *combiners* detectará que Alice ha comprado en 3 tiendas distintas en total**. Así que Alice podría no ser emitida al reducer o no contar correctamente.

El fallo ocurre porque el **Combiner** está ejecutando una lógica que no es segura para un **procesamiento distribuido** en este contexto.

Un *Combiner* debería realizar operaciones de **reducción local que sean conmutativas y asociativas**, y que puedan recombinarse correctamente en el Reducer final. Pero en este caso, el Combiner calcula localmente para cada persona la lista de tiendas distintas en las que compró, y decide en ese momento si esa persona ha comprado en 3 o más tiendas diferentes, emitiendo su nombre si cumple la condición.

Como hemos visto en el ejemplo cuando estos resultados parciales lleguen al *Reducer*, ya **no hay forma de saber cuántas tiendas distintas tenía Alice en total** porque los *combiners* no transmitieron el conjunto completo de tiendas.

Por lo tanto el problema es que cada Combiner decide prematuramente y de forma local si una persona cumple o no la condición, sin tener toda la información global disponible.

Y como en producción los *combiners* se pueden ejecutar de forma diferente en cada corrida (porque Hadoop decide cómo particionar los datos), a veces el error aparece y a veces no

Para corregir este defecto, el equipo de desarrollo debería:

- Evitar que el **Combiner** tome decisiones finales sobre el criterio de 3 o más tiendas.
- Diseñar la salida del Mapper de modo que emita datos que puedan combinarse de forma segura, sin perder información.

Recomendaciones concretas:

- Que el **Combiner** solo agrupe las compras de una persona y obtenga la lista parcial de **tiendas**, pero sin decidir si cumple la condición.

```
1  #!/usr/bin/python3
2
3  import sys
4
5  current_person = None
6  tiendas = set()
7
8  for line in sys.stdin:
9      person, tienda = line.strip().split("\t", 1)
10
11     if current_person != person:
12         if current_person:
13             tiendas_str = ",".join(tiendas)
14             print(f"{current_person}\t{tiendas_str}")
15             current_person = person
16             tiendas = set()
17
18     tiendas.add(tienda)
19
20 # Último
21 if current_person:
22     tiendas_str = ",".join(tiendas)
23     print(f"{current_person}\t{tiendas_str}")
```

- Y el *Reducer* final combine todas las listas de tiendas distintas por persona de todos los *combiners* y **sólo ahí decida si alcanza 3 o más**.

Alternativa aún más segura (Menos óptima): No usar *Combiner* en este caso, porque el beneficio es bajo y el riesgo de inconsistencia es alto si no se implementa con cuidado. Si se quiere mantener, asegurar que el *Combiner*:

- Solo agrupe tiendas por persona (sin condicionar)
- Y el Reducer final acumule las listas y decida.

¿Fue útil la información aportada?

Si, porque esa información nos indicó exactamente en qué escenario se producía el fallo, lo que permitió descartar otros posibles errores en *Mapper* o *Reducer*. La pista de que el defecto ocurría cuando había más de un *Combiner* nos llevó a analizar si la lógica del *Combiner* era segura en un entorno distribuido y efectivamente, vimos que no lo era porque tomaba decisiones parciales de forma local, sin esperar a tener todos los datos de una persona.

La imagen y el mensaje redujeron mucho el espacio de búsqueda del defecto.

Código Mapper

```
1  #!/usr/bin/python3
2
3  '''
4  Programa realizado por Jesus Moran
5  Este programa contiene un defecto que tienen que encontrar los alumnos
6  '''
7
8  import sys
9
10 for linea in sys.stdin:
11     linea = linea.strip()
12     cliente,tienda = linea.split("\t", 1)
13     print("%s\t%s" % (cliente, tienda))
14
```

Lee cada línea del *stdin* (que sería una línea del fichero .txt que has puesto), separa el valor en dos partes usando `split("\t", 1)`:

- **cliente** (nombre de la persona)
- **tienda** (nombre de la tienda)

Y luego **emite (imprime)** como salida: **Cliente** /t **Tienda**. Es decir, pasa los mismos datos, pero como key-value separados por tabulador

De esta forma tendremos una clave en formato Cliente y un valor con la Tienda:

Input:

- Alice Nun Corp
- Bob Neque Vitae Limited
- Carol Neque Vitae Limited
- Dave Cursus Diam Inc.

Output:

- Alice Nun Corp
- Bob Neque Vitae Limited
- Carol Neque Vitae Limited
- Dave Cursus Diam Inc.

Código Combiner

```

1  #!/usr/bin/python3
2
3  '''
4  Programa realizado por Jesus Moran
5  Este programa contiene un defecto que tienen que encontrar los alumnos
6  '''
7
8  import sys
9
10 subproblema = None
11 tiendas = set()
12
13 for claveValor in sys.stdin:
14     cliente, tienda = claveValor.strip().split("\t", 1)
15
16     if subproblema == None:
17         subproblema = cliente
18
19     if subproblema == cliente:
20         tiendas.add(tienda)
21
22     else:
23         if len(tiendas) >= 3:
24             for numTiendaVisitada in range(0, 3, 1):
25                 tiendaVisitada = tiendas.pop()
26                 print("%s\t%s" % (subproblema, tiendaVisitada))
27
28             subproblema = cliente
29             tiendas = set()
30             tiendas.add(tienda)
31
32
33 if len(tiendas) >= 3:
34     for numTiendaVisitada in range(0, 3, 1):
35         tiendaVisitada = tiendas.pop()
36         print("%s\t%s" % (subproblema, tiendaVisitada))
37

```

El script es un *combiner* de Hadoop escrito en Python, diseñado para procesar pares clave-valor que representan clientes y las tiendas que han visitado. Su objetivo es agrupar las tiendas por cliente y, cuando un cliente ha visitado al menos tres tiendas diferentes, emitir tres de esas tiendas para ese cliente.

El programa lee línea a línea desde la entrada estándar (stdin). Cada línea contiene un cliente y una tienda separados por tabulador. Para cada línea, el código extrae el cliente y la tienda, y mantiene un conjunto (set) de tiendas visitadas por el cliente actual, almacenado en la variable `subproblema`.

Cuando el cliente cambia (es decir, llega un nuevo cliente distinto al que estaba siendo procesado), el programa verifica si el cliente anterior ha visitado al menos tres tiendas. Si es así, extrae tres tiendas arbitrarias del conjunto usando `pop()` y las imprime como salida. Luego reinicia el conjunto para comenzar a acumular las tiendas del nuevo cliente. Al finalizar la lectura de todas las líneas, realiza esta misma verificación e impresión para el último cliente.

El conjunto tiendas se usa para evitar duplicados, asegurando que se consideran solo tiendas diferentes. Sin embargo, el uso de `pop()` para extraer tiendas hace que el orden sea no determinista y que las tiendas seleccionadas sean arbitrarias. Además, el *combiner* elimina estas tiendas del conjunto, lo que implica que solo tres tiendas se mantienen y las demás se descartan en esta etapa.

Este comportamiento es problemático en Hadoop porque el *combiner* se ejecuta localmente y no garantiza que todas las visitas sean consideradas al final del proceso, ya que puede aplicarse varias veces en nodos diferentes. Esto puede provocar pérdida aleatoria de registros y discrepancias en los resultados finales cuando se compara con una ejecución sin *combiner*.

Por tanto, el código contiene un defecto clave: no preserva la integridad ni la totalidad de los datos cuando se fuerza el uso del *combiner*, debido al manejo inapropiado del conjunto y la eliminación de elementos con `pop()`.

Input:

- Alice Nunc Corp.
- Alice Nunc Sed Corp.
- Alice Arcu Aliquam Company
- Alice Pharetra Quisque Ac Company
- Bob Neque Vitae Limited
- Bob Neque Vitae Limited
- Bob Neque Vitae Limited
- Carol Neque Vitae Limited
- Carol Neque Vitae Limited
- Carol Mi Lorem Inc
- Dave Cursus Diam Inc.
- Dave Vestibulum Neque LLP
- Dave Orci Tincidunt Incorporated
- Dave Montes Nascetur LLP

Output esperado:

- Alice Nam Ac PC
- Alice Nisi Cum Sociis LLC
- Alice Pharetra Quisque Ac Company
- Dave Orci Tincidunt Incorporated
- Dave Id Sapien Cras Associates
- Dave Vestibulum Neque LLP

Alice tiene muchas tiendas diferentes, el combiner elige arbitrariamente 3 de ellas (`pop` del set no tiene orden).

Dave también tiene más de 3 tiendas distintas, por eso imprime 3 tiendas.

Cientes como Bob o Carol no aparecen porque Bob siempre tiene la misma tienda ("Neque Vitae Limited") y Carol también, por lo que no llegan a 3 tiendas distintas.

Código Reducer

```
1  #!/usr/bin/python3
2
3  '''
4  Programa realizado por Jesus Moran
5  Este programa contiene un defecto que tienen que encontrar los alumnos
6  '''
7
8  import sys
9
10 subproblema = None
11 tiendas = set()
12
13 for claveValor in sys.stdin:
14     cliente, tienda = claveValor.strip().split("\t", 1)
15
16     if subproblema == None:
17         subproblema = cliente
18
19     if subproblema == cliente:
20         tiendas.add(tienda)
21
22     else:
23         if len(tiendas) >= 3:
24             print("%s" % (subproblema))
25
26             subproblema = cliente
27             tiendas = set()
28             tiendas.add(tienda)
29
30
31 if len(tiendas) >= 3:
32     print("%s" % (subproblema))
33
```

Este script es un *reducer* en Python para un proceso de Hadoop que recibe pares clave-valor desde la entrada estándar, donde la clave es un cliente y el valor es una tienda visitada. El objetivo del *reducer* es identificar qué clientes han visitado al menos tres tiendas diferentes.

El programa lee línea por línea, extrayendo el cliente y la tienda de cada entrada. Mantiene una variable *subproblema* que almacena el cliente actual que se está procesando, y un conjunto (*set*) llamado *tiendas* para acumular las tiendas únicas visitadas por ese cliente.

Cuando el script detecta que la línea corresponde al mismo cliente (*subproblema == cliente*), añade la tienda al conjunto, evitando duplicados. Si cambia el cliente, el programa verifica si el

cliente anterior tiene tres o más tiendas diferentes. En ese caso, imprime el nombre del cliente como salida, indicando que cumple el criterio.

Luego, reinicia el conjunto para el nuevo cliente y continúa el proceso. Al finalizar la lectura, también verifica el último cliente para asegurar que su resultado se imprima si cumple la condición.

El defecto principal es que no imprime las tiendas visitadas, sólo el cliente.

Input:

- Alice Nam Ac PC
- Alice Nisi Cum Sociis LLC
- Alice Pharetra Quisque Ac Company
- Dave Orci Tincidunt Incorporated
- Dave Id Sapien Cras Associates
- Dave Vestibulum Neque LLP

Output

- Alice
- Dave

Ejecución con y sin defecto

Ejecutamos de forma natural el job en pseudo-distribuido:

```
dani@dani-VMware-Virtual-Platform:~$ hadoop jar /home/dani/Escritorio/hadoop/hadoop-3.3.6/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -file /home/dani/pruebafallo/mapperPersonasQueCompranEnMuchasTiendas.py -mapper /home/dani/pruebafallo/mapperPersonasQueCompranEnMuchasTiendas.py -file /home/dani/pruebafallo/combinerPersonasQueCompranEnMuchasTiendas.py -combiner /home/dani/pruebafallo/combinerPersonasQueCompranEnMuchasTiendas.py -file /home/dani/pruebafallo/reducerPersonasQueCompranEnMuchasTiendas.py -reducer /home/dani/pruebafallo/reducerPersonasQueCompranEnMuchasTiendas.py -input /user/dani/pruebafallo/casoDePrueba.txt -output /user/dani/pruebafallo/salidaPersonasQueCompranEnMuchasTiendas
```

Map-Reduce Framework

```
Map input records=104
Map output records=104
Map output bytes=2646
Map output materialized bytes=429
Input split bytes=224
Combine input records=104
Combine output records=15
Reduce input groups=3
Reduce shuffle bytes=429
Reduce input records=15
Reduce output records=3
Spilled Records=30
Shuffled Maps =2
Failed Shuffles=0
Merged Map outputs=2
GC time elapsed (ms)=76
CPU time spent (ms)=3120
Physical memory (bytes) snapshot=965267456
Virtual memory (bytes) snapshot=8181960704
Total committed heap usage (bytes)=980418560
Peak Map Physical memory (bytes)=385060864
Peak Map Virtual memory (bytes)=2726342656
Peak Reduce Physical memory (bytes)=241377280
```

Los contadores reflejan que la ejecución del job se desarrolló correctamente y que el flujo de datos entre las fases de **map**, **combine** y **reduce** funcionó como se esperaba.

- **Map input records = 104** y **Map output records = 104** indican que se procesaron 104 registros en la fase de mapeo y cada uno produjo una salida, sin pérdidas o filtrados inesperados.
- La diferencia entre **Map output bytes (2646)** y **Map output materialized bytes (429)** sugiere que el proceso de materialización (almacenamiento temporal para shuffle) redujo el tamaño de datos, probablemente porque el combine agrupa o compacta información antes del shuffle, lo que es eficiente para reducir tráfico de red.

- **Combine input records = 104** y **Combine output records = 15** evidencian que el combinador realizó una buena agregación local, reduciendo de 104 a 15 registros que se enviaron al reduce.
- En la fase de reducción, **Reduce input groups = 3** y **Reduce input records = 15** indican que se formaron 3 grupos principales (claves distintas) con 15 registros totales para procesar.
- Finalmente, **Reduce output records = 3** muestra que la reducción produjo 3 resultados finales, acorde con el número de grupos, probablemente el resultado esperado.

```
dani@dani-VMware-Virtual-Platform:~$ hdfs dfs -cat /user/dani/pruebafallo/salida
PersonasQueCompranEnMuchasTiendas/part-00000
Alice
Carol
Dave
```

Para reproducir el fallo en el job de Hadoop, se hicieron estos pasos clave:

1. **Forzar el uso del combiner:** Se configuró el job para que siempre utilice el combiner. Esto implica que la función del combiner se ejecuta localmente en los nodos map antes de enviar los datos al reducer.
2. **Trabajar con particiones pequeñas:** Se usaron pocos splits o particiones con datos reducidos. Esto aumenta la probabilidad de que el combiner procese sólo subconjuntos parciales de datos por cliente.
3. **Datos con múltiples ocurrencias por clave (cliente):** Los datos de entrada contenían clientes con muchas tiendas distintas, pero divididas entre varios splits.
4. **Ejecutar y comparar resultados:** Se ejecutó el job con y sin combiner para comparar la salida final.

```
dani@dani-VMware-Virtual-Platform:~$ hadoop jar /home/dani/Escritorio/hadoop/hadoop-3.3.6/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
-file /home/dani/pruebafallo/mapperPersonasQueCompranEnMuchasTiendas.py \
-mapper /home/dani/pruebafallo/mapperPersonasQueCompranEnMuchasTiendas.py \
-file /home/dani/pruebafallo/combinerPersonasQueCompranEnMuchasTiendas.py \
-combiner /home/dani/pruebafallo/combinerPersonasQueCompranEnMuchasTiendas.py \
-file /home/dani/pruebafallo/reducerPersonasQueCompranEnMuchasTiendas.py \
-reducer /home/dani/pruebafallo/reducerPersonasQueCompranEnMuchasTiendas.py \
-input /user/dani/pruebafallo/splits \
-output /user/dani/pruebafallo/salidaPersonasQueCompranEnMuchasTiendasDefecto
```

```

Map-Reduce Framework
  Map input records=104
  Map output records=104
  Map output bytes=2646
  Map output materialized bytes=818
  Input split bytes=1210
  Combine input records=104
  Combine output records=27
  Reduce input groups=2
  Reduce shuffle bytes=818
  Reduce input records=27
  Reduce output records=2
  Spilled Records=54
  Shuffled Maps =11
  Failed Shuffles=0
  Merged Map outputs=11
  GC time elapsed (ms)=1008
  CPU time spent (ms)=10460
  Physical memory (bytes) snapshot=3952230400
  Virtual memory (bytes) snapshot=32757604352
  Total committed heap usage (bytes)=3741319168

```

Reduce input groups y reduce output records:

- Sin fallo: 3 grupos de reduce, 3 registros de salida.
- Con fallo: 2 grupos de reduce, 2 registros de salida.

Aquí se puede notar que el número de grupos que reduce procesa es menor en la segunda ejecución (2 vs 3), y produce menos registros de salida.

Combine output records y Reduce input records:

- Sin fallo: combine reduce 15 registros, reduce recibe 15 registros.
- Con fallo: combine 27 registros, reduce recibe 27 registros.

Esto puede indicar que el combinador está enviando más registros al reduce que antes, lo que puede afectar la lógica de reducción (podría estar generando duplicados o agrupaciones incorrectas).

```

dani@dani-VMware-Virtual-Platform:~$ hdfs dfs -cat /user/dani/pruebafallo/salida
PersonasQueCompranEnMuchasTiendasDefecto/part-00000
Alice
Dave

```